

09 · Paralleles Rechnen (Multicore) & Finale

Mikrocomputertechnik 3 · Von SMP, MESI-Kohärenz und Hardware-Locks zum fertigen SoC

Prof. Dr.-Ing. Daniel Klünder

Folie

1 / 51

Willkommen zum Finale!

- Neunte und letzte Vorlesung in Mikrocomputertechnik 3
- Weg: Speicherzelle → Caches → DMA → Busse → OoO → Edge AI
- Heute: Multi-Core — Thread-Level Parallelism auf dem SoC
- Praxis: Theorie zu MESI/Atomics, danach Wrap-Up und Klausurvorbereitung

Notizen

Letzter Baustein: zweiter (und weiterer) Prozessorkern — und wie Software damit sicher umgeht.

Folie

2 / 51

Agenda und Lernziele

1. Cache-Kohärenz und MESI (Bus Snooping)
2. Race Conditions und Hardware-Atomics (LR/SC, AMO)
3. Amdahls Gesetz und Semester-Rückblick / Klausurfokus

Lernziele:

- Kohärenzproblem und MESI-Zustände erklären
- Race Conditions auf Assembler-Ebene begründen
- Software-Locks vs. Hardware-Atomics unterscheiden
- Speedup-Grenzen mit Amdahl abschätzen
- Systemwissen der Blöcke 1–8 für die Klausur strukturieren

Notizen

Zwei Probleme: Hardware-Kohärenz der Caches; Software-Atomizität bei Shared-Updates.

Anknüpfung: Edge-AI-Labor

Kurz-Recap aus Block 8 (Vertiefung im Übungsblatt):

- TFLM-Inferenz auf dem RISC-V-Kern
- Matrix-Last blockiert die General-Purpose-CPU lange
- Eine funktionale Simulation hat keine Memory Wall gemessen
- TLP und DLP schließen sich nicht aus

Notizen

OS-Tasks sind ein TLP-Beispiel. Auch Datenarbeit kann Threads nutzen. Matrizen können zusätzlich DLP verwenden.

Erinnerung: Taktraten-Mauer

- Warum Multicore? Power Wall (Block 7)
- Höherer Takt → stark steigende Verlustleistung und Kühlbedarf
- Besonders kritisch bei Mobile/Embedded
- Ausweg: mehr Kerne statt immer höherer GHz

Notizen

Plan B der Industrie nach dem GHz-Wettrennen: Parallelität auf Thread-Ebene.

Multi-Core: Thread-Level Parallelism

- Zwei (oder mehr) Kerne auf einem Chip statt „doppelt so schneller“ Einzelkern
- TLP: unabhängige Threads/Tasks parallel
- Beispiel: UI/Media auf Kern 0, Hintergrundarbeit auf Kern 1
- Mehr Kerne sparen nicht stets Energie und beschleunigen einen einzelnen Thread nicht

Notizen

Zweiter Arbeiter statt einen zu überhitzen — aber Kommunikation wird zum Problem.

Symmetric Multiprocessing (SMP)

SMP-Lehrmodell: gleiche Kerne, ein OS, gemeinsamer Speicher. L2 ist Beispiel, keine Definition.



Migration nur, wenn Scheduler, Port und Affinity es erlauben.

Engpaasse: Interconnect, L2, DRAM, Locks. Mehr Kerne beschleunigen einen Thread nicht automatisch.

SMP: ein OS, gleicher Zugriff. AMP: getrennte Software, oft ohne diese Kohärenz.

FreeRTOS-SMP gilt nur fuer passende Ports und Konfigurationen.

Heterogene Kerne sind hier ausgeblendet.

TLP kann Aufgaben und Datenarbeit betreffen. DLP bleibt zusätzlich möglich.

Block 8 hat keine gemessene Memory Wall aus Renode geliefert.

Abbildung 1: Zwei gleiche Kerne, private L1, beispielhaftes L2. Migration ist nicht automatisch.

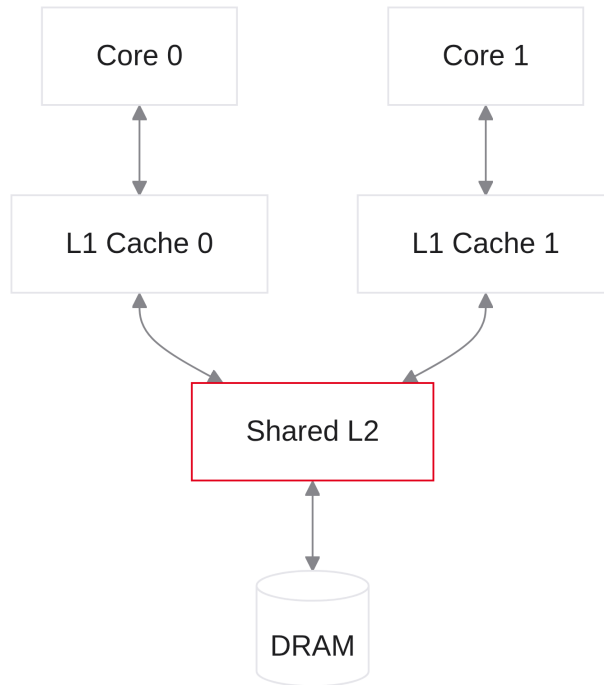
- Lehr-SMP: gleiche Kerne, ein Betriebssystem, gemeinsamer Speicherzugriff
- Private L1 und ein geteiltes L2 sind ein Beispiel, keine Definition
- Migration hängt von Scheduler, Port und Affinity ab. Nicht jedes FreeRTOS ist SMP
- AMP und heterogene Kerne bleiben abgegrenzt

Notizen

Klassisches PC-/Phone-Modell: gleichberechtigte Kerne, Scheduler verteilt.

Architektur eines SMP-SoC

- Pro Kern: privater L1 (I und D)
- Geteilt: oft L2 und DRAM (Shared Memory)



Notizen

Private L1 für Tempo — darunter treffen sich die Kerne. Dort lauert die Kohärenzfrage.

Die Leitfrage des Tages

- Write-Back (Block 2): Writes oft nur im privaten L1 — RAM kann veraltet sein
- Core 0 schreibt `score = 10` in seinen L1
- Core 1 will `score` lesen
- Woher weiß Core 1, dass die Wahrheit im L1 von Core 0 liegt — nicht im alten RAM?

Notizen

Eine CPU: Write-Back genial. Zwei CPUs: ohne Kohärenz zerbricht die Semantik.

Kohärenzverlust (Stale Data)

Kohaerenz, Konsistenz und Atomizität sind drei Fragen.

Write-Through ohne Invalidation RAM wird 5. Core1-L1 kann weiter 0 halten.		Koordinierte Kohärenz Fremde Kopie wird ungültig oder mit 5 versorgt.
MESI	Writes einer Adresse	kein ++ und kein Flag
Atomic-RMW	ein RMW unteilbar	keine Publikationsordnung
Acquire/Release	Ordnung um ein Datum	keine Kohärenz allein

Ohne Hardware-Kohärenz bleiben non-cacheable Speicher und Ownership möglich, oft in AMP.

Für cachebares SMP-Sharing ist das aufwendig, DMA-Maintenance ist nicht selten per Definition.

Nicht jede RAM-Kopie muss in jedem Takt den neuesten Wert halten.

Abbildung 2: Write-Through ohne Invalidation lässt die fremde Kopie stehen. MESI, RMW und Ordnung lösen Verschiedenes.

- Private Write-Back-Caches können unterschiedliche Sichten auf dieselbe Adresse erzeugen
- Kohärenz (pro Adresse): alle Kerne sehen eine einheitliche Reihenfolge der Writes
- Kohärenz ordnet Writes einer Adresse. Sie macht weder `i++` atomar noch ein Daten-plus-Flag korrekt
- Non-cacheable Speicher und Ownership bleiben möglich, besonders in AMP

Notizen

Konsistenz beschreibt Beobachtungsreihenfolgen mehrerer Zugriffe. Atomizität schützt eine Operation vor Teilbeobachtung.

Warum Software-Flush scheitert

- Für cachebares SMP-Sharing ist manuelles Spülen nach jedem Write unpraktikabel
- DMA-Maintenance ist nicht per Definition selten
- Hardware-Kohärenz kann für die Anwendung unsichtbar sein und kostet trotzdem Traffic

Notizen

Write-Through aktualisiert den unteren Speicher und invalidiert allein keine fremde Cache-Kopie.

Bus Snooping

- Klassisch: Cache-Controller lauschen am gemeinsamen Bus/Interconnect
- Fremder Zugriff auf lokal Modified → eingreifen (Daten liefern / Invalidate)
- Große Systeme oft directory-basiert statt reinem Broadcast-Snoop

Notizen

Idee bleibt: Caches koordinieren die „Wahrheit“ einer Adresse — Implementierung variiert.

MESI-Protokoll

- Zustandsautomat je Cache-Line. 64 Byte sind hier nur ein Beispiel
- Zustände: Modified (M), Exclusive (E), Shared (S), Invalid (I)
- Basis vieler kohärenter Multicore-Designs (Varianten: MOESI, ...)

Notizen

Konzept der vier Buchstaben zählt — nicht jede Transitionstabelle auswendig.

MESI: Zustandsübersicht

Vereinfachtes Write-Back-MESI. Transiente Zustände ausgeblendet. 64 B nur Beispiel.



I + lokaler Read, keine Kopie -> E, Speicher liefert

I + lokaler Read, andere saubere Kopie -> S

I + lokaler Read, Owner M -> Daten vom Owner, dann S

E + lokaler Write -> M, still

E + Fremdread -> S, Daten teilen

S + eigener Write -> erst Ownership, Invalidates, dann M

S + Fremdwrite -> I

M + Fremdread -> aktuelle Daten liefern, dann S

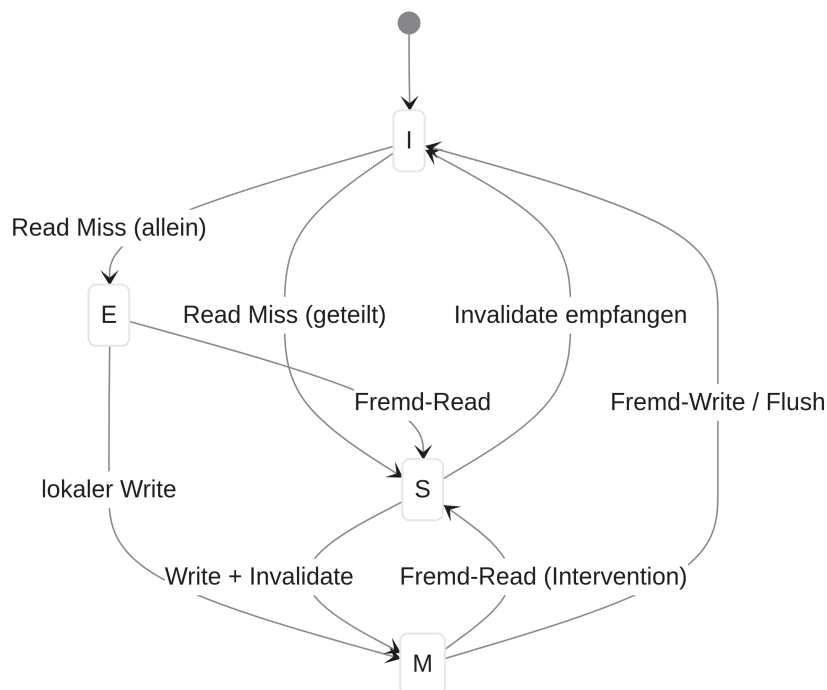
M + Fremdwrite -> Daten liefern, dann I

Eviction von M schreibt zurück. Eviction von S kann S woanders lassen.

S heisst sauber und teilbar, nicht nachweislich mehrere Kopien.

I heisst lokal ungültig, nicht zwingend gelöschte Bits. M und E sind exklusiv.

Abbildung 3: Lokale Ereignisse und Fremdtransaktionen getrennt. S bedeutet teilbar, nicht zwingend mehrere Kopien.



Notizen

Die Pfeile sind die klausurrelevanten Übergänge. Merken Sie sich besonders: Write auf Shared braucht Invalidate; Read-Miss auf Modified liefert der Owner.

Zustand M (Modified)

- Line lokal geändert. Der untere Stand kann noch alt sein
- Exklusiv: kein anderer gültiger Besitzer im Modell
- Klassischer Write-Back-„Boss“-Zustand

Notizen

Monopol auf die Wahrheit dieser Adresse.

Zustand E (Exclusive)

- Sauber: Übereinstimmung mit dem kohärenten unteren Stand
- Exklusiv: niemand sonst hat die Line
- Write → oft still zu M, ohne Broadcast

Notizen

Sauber und allein — Schreiben ohne Netzwerk-Meeting.

Zustand S (Shared)

- Sauber und teilbar. Es müssen nicht nachweislich mehrere Kopien existieren
- Gut für paralleles Lesen
- Vor lokalem Write: andere Kopien invalidieren

Notizen

Read-only-Sharing effizient — Write braucht Kohärenzaktion.

Zustand I (Invalid)

- Nach Reset oder Invalidate
- Lokal ungültig. Die Bits müssen nicht gelöscht sein. Ein Hit auf I ist verboten
- Daten müssen neu geholt werden

Notizen

Leerer Eimer — nicht verwenden.

MESI: Read-Miss

Core 0 liest x , lokal I:

- Request auf dem Interconnect
- Niemand hat x → Memory liefert, Core 0 oft → E
- Anderer Kern hat x sauber (E/S) → beide → S
- Anderer Kern hat x Modified → der Owner liefert (Write-Back/Intervention), Zustände typisch → S

Notizen

Snooping/Directory entscheidet: allein, geteilt oder Owner-Supply bei Modified.

MESI: Write auf Shared

S nach M ist mehr als ein Broadcast. Erst Ownership und Bestätigung, dann der Write.



Fremdread einer M-Line: Owner liefert 5, wird S, Empfänger wird S, RAM wird 5.

Dirty-Shared oder Owned gehören nicht zu diesem MESI.

Ein Broadcast ohne abgeschlossene Invalidates ist noch kein Write.

Studierende prüfen die Tabelle: nach Schritt 3 ist RAM noch 0, nach Schritt 4 ist er 5.

Mehrstufige Hierarchien sind hier auf L1 plus unteren Stand vereinfacht.

Sauber heisst: Übereinstimmung mit dem kohärenten unteren Stand, nicht magisch mit jedem RAM-Takt.

Abbildung 4: Erst Ownership und Bestätigung, dann der Write. Ein Broadcast allein ist noch kein Abschluss.

- Core 0 und 1: x in S
- Core 0: $x = 5$
- Erst Ownership und die nötigen Invalidates abschließen
- Danach im Modell schreiben und nach M gehen. Der Broadcast allein ist nicht der Abschluss

Notizen

Geteilte Ressource nie heimlich ändern.

Reaktion: Invalidate

- Core 1 sieht Invalidate für $x \rightarrow$ lokal I
- Späteres Lesen $\rightarrow$ Miss $\rightarrow$ frische Kopie (Wert 5) von Core 0 / Memory-Pfad

Notizen

Konsistenz: erzwungenes Verwerfen, dann Nachladen der Wahrheit.

MESI-Beispiel: Zustandsfolge

x startet 0, beide I, RAM 0. Klassischer MESI-Pfad, Write-Back.

Schritt	Core0	Core1	RAM	Aktion
0 Read	E:0	I	0	Speicher liefert
1 Read	S:0	S:0	0	E wird S
0 Write 5	M:5	I	0	Invalidate, RAM noch 0
1 Read	S:5	S:5	5	Owner liefert 5

Bei M darf der untere Speicher noch 0 sein. Der folgende Read muss 5 sehen.

Invariante: höchstens ein M oder E. S-Kopien stimmen überein. I ist kein Hit.

Gleichzeitige Schreiber: der Interconnect serialisiert Ownership.

Snooping ist nicht an einen einzelnen Draht gebunden. Directory nur als Abgrenzung.

Abbildung 5: Nach dem Write ist Core0 M mit 5 und der RAM noch 0. Der nächste Read liefert 5.

Zwei Kerne, Variable x, Start beide Invalid:

Schritt	Aktion	Core 0	Core 1
1	Core 0 liest x	E	I
2	Core 1 liest x	S	S
3	Core 0 schreibt x=5	M	I
4	Core 1 liest x	S	S

- Nach dem Write kann der RAM noch 0 enthalten, während Core0 M den Wert 5 hält
- Der folgende Read erhält 5. Beide Lines werden S, der untere Stand wird 5

Notizen

Typische Klausurfrage: Tabelle ausfüllen. Üben Sie die Sequenz Read-Miss → Shared → Write → Invalidate → erneutes Lesen.

Overhead von MESI

- Preis: Kohärenz-Traffic (Invalidate, Snoop)
- Viele Kerne + häufige Shared-Writes → Interconnect belastet
- Große Server: Ringe/Meshes statt einfachem Broadcast-Bus

Notizen

Skalierung: Verwaltungslast kann Datenbandbreite fressen.

False Sharing

Line beispielhaft 64 B, Basis 0x20001000. Keine Zeitmessung.

False Sharing

temp1 bei +0, temp2 bei +4, gleiche Line

Getrennt

temp2 bei +64 = 0x20001040

Jeder Write holt die Ownership der ganzen Line. Der andere Kern verliert sie.

True Sharing wäre ein gemeinsamer Zähler, nicht zwei eigene Variablen.

Padding hilft nur mit Basis, Grösse und echter Line. Array-Alignment allein reicht nicht.

Preis: mehr Speicher und manchmal schlechtere Lokalität.

Messung: Threads pinnen, Arbeit gleich, mehrfach, Optimierung nicht wegwerfen.

Ein Simulator ohne Kohärenzzeit belegt keine Verlangsamung.

Abbildung 6: temp1 bei +0 und temp2 bei +4 teilen die 64-Byte-Line ab 0x20001000.

- Core 0: temp1, Core 1: temp2 — logisch unabhängig
- Liegen in derselben Cache-Line → Write invalidiert den Partner
- Ping-Pong trotz verschiedener Variablen
- Padding trennt nur bei passender Basis, Objektgröße und echter Line
- Es kostet Speicher und kann die Lokalität verschlechtern. Ein Simulator ohne Kohärenzzeit misst das nicht

Notizen

64 Byte sind die angenommene Line, keine universelle Größe. True Sharing wäre der gemeinsame Zähler.

Aufgabe — MESI-Zeile

Core0 hat nach seinem Write den Zustand M und den Wert 5. Welchen Wert hat der RAM in diesem Modell noch, und was sieht Core1 beim folgenden Read?

Notizen

Der RAM kann noch 0 sein. Core1 erhält 5, beide werden S, der untere Stand wird 5. S heißt nicht, dass vorher mehrere Kopien nachgewiesen wurden. Siehe die Zustandsfolge.

Software: gemeinsamer Zähler

- MESI: Caches *kohärent* (pro Adresse)
- Semantik von `zaehler++` (RMW) ist MESI egal
- Beide Kerne: `zaehler++` auf globalem `int` — Erwartung: `+2`

Notizen

Einfachste Aufgabe — parallel gefährlich.

Anatomie von `i++`

Eine Quelltextoperation, keine Atomizität. Das typische RMW lautet:

```
lw    t0, 0(s1)    # Read
addi  t0, t0, 1    # Modify
sw    t0, 0(s1)    # Write
```

- Drei Schritte — können sich zwischen Kernen überlappen

Notizen

Dieses Interleaving motiviert das Problem. Ein konkurrierendes normales C-int ist ein Data Race und in C11 undefiniert, nicht nur manchmal 1.

Race Condition

Start 0. Gewöhnliche Writes bleiben kohärent und verlieren trotzdem ein Update.

Core0 Read 0 Add 1 Write 1

Core1 Read 0 Add 1 Write 1

Speicher endet bei 1. MESI hat beide Writes geordnet, die Rechnung nicht.

Zwei atomare Fetch-Adds: alte Werte 0 und 1, Ende 2. Reihenfolge der Alten offen.

Ungeschütztes C-int ist ein Data Race und in C11 undefiniert. Das Bild ist nur Motivation.

i++ ist eine Quelltextoperation, keine garantierte Atomic-RMW.

AMO kann das RMW in einer Instruktion ausdrücken. Intern bleibt ein Datenpfad.

volatile erzwingt Compilerzugriffe und schafft weder Atomizität noch Ordnung.

Abbildung 7: Beide schreiben 1. MESI verbreitet die Writes, der Endwert bleibt 1.

1. Core 0 liest 0
2. Core 1 liest 0 (Shared)
3. Core 0 schreibt 1
4. Core 1 schreibt ebenfalls 1 (seine lokale Rechnung)

Im Hardwarebild endet der Speicher bei 1. Das ist keine garantierte Semantik des fehlerhaften C-Programms.

Notizen

Wettlauf: beide rechnen auf demselben Startwert.

Naive Software-Locks

```
/* ohne volatile: Compiler darf den Spin-Loop wegoptimieren */
while (lock == 1) { }
lock = 1;
zaehler++;
lock = 0;
```

Idee: Mutual Exclusion — aber Check und Set sind getrennt.

Notizen

Schloss-Metapher stimmt logisch — atomar nicht. `volatile` verhindert Compiler-Optimierung des Spins, macht Check+Set aber nicht atomar.

Warum diese Check-then-Set-Schleife scheitert

- Prüfen und Setzen sind zwei Operationen. `volatile` ändert das nicht
- Beide können frei sehen und eintreten
- Korrekte Software-Locks nutzen atomare Primitive und Ordnung, sie sind nicht unmöglich

Notizen

Rein algorithmische Verfahren bleiben eine angenommene, fortgeschrittene Alternative.

RISC-V A-Extension

- A oder Zlrsc/Zaamo, ausgerichtet, nur in unterstützten Regionen
- LR/SC und AMO gelten für eine Adresse in ihrer Domäne, nicht für den ganzen Bus
- DMA gehört nur dazu, wenn die Reservationsdomäne das festlegt

Notizen

Ohne Atomics kein korrektes Multicore-OS auf Shared Memory. DMA braucht eigene Sync-Disziplin.

LR/SC

- `lr.w` liest und setzt eine Reservation auf eine implementierungsabhängige Granule
- `sc.w` schreibt bei Erfolg und liefert 0. Bei Fehlschlag kein Store und ein Wert ungleich 0
- Die Reservation ist kein gehaltenes Mutex

Notizen

Überwachsungszaun um die Adresse zwischen Load und Store.

LR/SC: Erfolg

- Core 0: `lr.w` → frei → `sc.w` erfolgreich
- Lock exklusiv übernommen

Notizen

Erfolgspfad: wie RMW, Monitor still.

LR/SC: Fehlschlag

LR setzt eine Reservation auf eine Granule, kein gehaltenes Mutex. SC-Erfolg liefert 0.

Hart	Gelesen	Reservation	SC	Speicher
0 LR	0	gesetzt	-	0
1 LR	0	gesetzt	-	0
0 SC	0	endet	0	1
1 SC	0	ungueltig	nicht 0	1
1 Retry	1	neu	0	2

Ein SC darf auch scheitern, wenn kein anderer Hart geschrieben hat.

Interrupt, Kontextwechsel und fremder Store koennen die Reservation loeschen.

DMA gehoert nur dazu, wenn die Plattform dieselbe Reservationsdomaene definiert.

Constrained Loops haben Fortschritt unter Spec-Bedingungen, keine unbegrenzte Fairness.

Abbildung 8: SC-Erfolg ist 0. Der zweite SC schreibt nicht. Ein Fehlschlag ohne fremden Store ist zulässig.

- Beide: `lr.w` nahezu gleichzeitig
- Core 0: `sc.w` ok
- Core 1: `sc.w` scheitert (Reservation gebrochen / Kohärenz)
- Software von Core 1 wiederholt die Schleife

Notizen

Zweiter Kern prallt ab — Zustand bleibt konsistent.

LR/SC: Spinlock in Assembler

Lehrschloss fuer gewoehnlichen Shared Memory. Kein Cache-Flush als Ersatz.

lr.w.aq pruefen sc.w bei Erfolg 0

kritische Daten liegen zwischen Acquire und Release

fence rw,w dann sw zero auf das Schloss

Acquire begrenzt spaetere Zugriffe, Release begrenzt fruehere Schreibzugriffe.

Daten plus Flag: der atomare Flag-Store allein publiziert die Daten nicht.

Spinlock: nicht schlafen, nicht blockierend warten, nicht lang rechnen.

ISR auf ein Schloss des unterbrochenen Kontexts: Selbstdeadlock.

Timeout bricht das Schloss nicht sicher, solange der Besitzer noch schreibt.

MMIO und Compiler-Barrieren sind andere Domaenen. Annahme: RVWMO, A-Extension.

Diese Sequenz ist hier nicht mit einer RISC-V-Toolchain assembliert.

Abbildung 9: Acquire mit lr.w.aq, Release mit fence rw,w vor dem Store. Kein Flush als Ersatz.

```
acquire:
    lr.w.aq t0, (a0)
    bnez    t0, acquire
    li      t1, 1
    sc.w    t2, t1, (a0)
    bnez    t2, acquire
release:
    fence  rw, w
    sw     zero, 0(a0)
```

- Erfolg von `sc.w` ist 0. Die Sequenz ist Lehrannahme für gewöhnlichen Speicher und hier nicht assembliert
- Im kritischen Abschnitt nicht schlafen und keine blockierende Arbeit ausführen

Notizen

Ohne die Release-Fence publiziert der Store das Schloss, nicht zuverlässig die vorherigen Daten. Ein ISR-Spin auf dasselbe Schloss kann sich selbst blockieren.

AMO-Befehle

`atomic_fetch_add` ist ohne Ordnung sequentially consistent. Das serialisiert nicht das ganze Programm.

Ziel kann AMO, LR/SC oder ein Library-Fallback sein. ABI und Typ entscheiden.

`amoadd.w`: Speicher 10, Addend 3. Rd bekommt 10, Speicher wird 13.

Zwei Fetch-Adds um 1 ab 0: alte Werte 0 und 1, Ende 2.

Relaxed genuegt fuer einen reinen Zaehler ohne weiteres publiziertes Datum.

Ein ISR darf keinen blockierenden Lock-Fallback rufen. Das ist eine eigene Pruefung.

Testidee: zwei Harts, je N Inkremente, Ende 2N. Breite ohne Ueberlauf.

Der normale C-Zaehler ist ein Data Race, keine Messreferenz.

AMO schuetzt eine Adresse in ihrer Domaene, nicht den Bus und nicht alle Daten.

DMA nimmt nicht automatisch am C11-Happens-Before teil.

Host-GCC ist kein Nachweis der RISC-V-Codegenerierung.

Abbildung 10: `amoadd.w` von 10 um 3 liefert alt 10 und neu 13. Zwei Inkremente enden bei 2.

- `amoadd.w` auf den Wert 10 mit Addend 3: Rückgabe 10, Speicher 13
- Das ist kein globaler Bus-Lock. Ohne `aq/rl` fehlt die zusätzliche Ordnung

Notizen

Ein Befehl statt explizitem LR/SC-Loop für einfache Ops.

C11: `<stdatomic.h>`

```
#include <stdatomic.h>

_Atomic int zaehler = 0;

void hart_increment(void) {
    atomic_fetch_add(&zaehler, 1); /* seq_cst */
}
```

- Der Compiler kann AMO, LR/SC oder einen Library-Fallback erzeugen
- `atomic_fetch_add` ist standardmäßig sequentially consistent und macht nicht das ganze Programm seriell
- Ein blockierender Fallback gehört nicht in eine ISR

Notizen

Modernes C trifft RISC-V-A — Brücke Software/Hardware.

Deadlocks

Deadlock ist ein Wartezyklus. Starvation ist Warten ohne Zyklus. Livelock ist Betrieb ohne Fortschritt.

Zyklus

Core0 haelt A, will B

Core1 haelt B, will A

Ordnung A vor B

beide erwerben A, dann B

kein Zyklus

Timeout allein stellt keinen gueltigen Datenzustand her.

Frage: Die ISR spinnt auf ein Schloss, das der unterbrochene Kern haelt. Was folgt?

Antwort in den Notizen: Selbstdeadlock. Atomics ersetzen keine Prioritaetsvererbung.

Das Schloss nicht blind freigeben, waehrend der Besitzer noch schreibt.

Abbildung 11: A-dann-B vermeidet den Zyklus. Timeout gibt keinen konsistenten Zustand zurueck.

- Locks gegen Races — Risiko: zirkulaeres Warten
- Core 0 haelt A, will B; Core 1 haelt B, will A
- Hardware rettet nicht — Reihenfolge der Lock-Hierarchie in Software

Notizen

Atomsics schuetzen Daten, nicht vor schlechter Lock-Ordnung.

Aufgabe — Schloss und ISR

Warum genuegt `volatile` nicht fuer das Check-then-Set? Was passiert, wenn die ISR auf das Schloss spinnt, das der unterbrochene Kontext haelt?

Notizen

`volatile` erzwingt Zugriffe und schafft keine Atomizitaet und keine Ordnung. Die ISR und der unterbrochene Kontext warten aufeinander: Selbstdeadlock. Muster: Acquire mit `lr.w.acq`, Erfolgscode 0, Release mit `fence rw, w` vor dem Store.

Amdahls Gesetz

- Nicht alles ist parallelisierbar: serieller Anteil s , paralleler Anteil $p = 1 - s$
- Speedup bei N Kernen:

$$S(N) = \frac{1}{s + \frac{p}{N}}$$

- Grenze für $N \rightarrow \infty$: $S_{\max} = 1/s$

Notizen

Mehr Kerne helfen nur dem parallelen Anteil. 10 % seriell $\rightarrow$ maximal 10 $\times$ Speedup — egal wie viele Kerne.

Amdahl: Zahlenbeispiel

Analytisch, fester Workload, kein Overhead. $s=0,2$ $p=0,8$.

$S(4) = 1/0,4 = 2,5$

$S(100) = 1/0,208 = 4,8077$

Grenze $1/s = 5$. Effizienz $S(100)/100 = 4,81$ Prozent.

4,8 liegt nahe an 5. Die Parallel-Effizienz ist trotzdem sehr klein.

Ideal ohne seriellen Anteil wäre $S(N)=N$. Die Kurve ist keine Messung.

Optional $T(N)/T(1) = s + p/N + o(N)$. Ohne genannte $o(N)$ bleibt der Overhead draussen.

Eine grössere Aufgabe ist eine andere Frage und widerlegt Amdahl nicht.

Annahmen: feste Arbeit, serieller Anteil der Ein-Kern-Zeit, ideale Balance.

Mehr DLP löst die Memory Wall nicht allein. Lokalität und Transfers bleiben.

Abbildung 12: $S(4)=2,5$ und $S(100)=4,8077$. Das liegt nahe an 5, die Effizienz ist etwa 4,8 Prozent.

- Programm: 80 % parallel, 20 % seriell ($s = 0,2$)
- 4 Kerne: $S(4) = 1/(0,2 + 0,8/4) = 1/0,4 = 2,5$
- 100 Kerne: $S(100) = 1/0,208 \approx 4,8077$, nahe an der Grenze 5
- Die Effizienz $S(100)/100$ ist nur etwa 4,8 Prozent. Overhead ist in dieser Rechnung noch 0

Notizen

Feste Arbeit, ideale Balance, kein $o(N)$. Eine größere Aufgabe ist eine andere Frage. 4,8 ist nicht weit unter 5.

Unser fertiges SoC (Rückblick)

Dual-Core-Idee mit Bausteinen aus MCT3:

- Konzept, nicht das Laborgerät: OoO, MESI und SMP sind hier nicht als vorhandenes Target belegt
- Behandelt: Caches, CLINT/PLIC als RISC-V-Beispiel, UART/SPI/I2C, DMA, Timer
- Optional und nicht im Repository nachgewiesen: RVV, NPU, kohärentes SMP

Notizen

Vom Addierer zum überblickbaren Microcontroller-SoC.

Drei Flaschenhälse — Lösungen

Feind	Lösungspfad
Memory Wall	Lokalität, Tiling, Bandbreite, nicht nur DLP
Control Hazards	Prediction, Unrolling, Scheduling
CPU blockiert durch I/O	Interrupts, DMA

Notizen

Optimierung heißt: welchen der drei Feinde treffe ich?

C-Code vs. Hardware

Ohne Hardwareblick typisch:

- Cache-feindliche Schleifen (Layout)
- Busy-Waiting auf UART/Status
- Shared `i++` ohne `_Atomic` / falsches `volatile`

Mit Systemblick: Caches, Pipeline, DMA und Kohärenz mitdenken.

Notizen

Röntgenblick: eine C-Zeile kann harmlos oder systemtötend sein.

Infos zur Klausur

- Dauer und Hilfsmittel nur aus der autorisierten Ankündigung. Hier nichts Neues festlegen
- Unbestätigte Formalia vor der Klausur prüfen
- Fokus der Vorlesung: Konzepte, kleine Rechnungen, Skizzen, kurze C-Ausschnitte

Notizen

Problemlösung vor Auswendiglernen von Datenblatt-Adressen.

Klausurfokus: Speicher

- AMAT bei Hit-Rate und Miss-Penalty
- Direct-Mapped vs. satzassoziativ; Thrashing
- Cache-Line: Tag, Index, Offset, Valid, Dirty
- Write-Back + DMA; MESI bei Multicore

Notizen

Adresse zerlegen und Hierarchiekosten rechnen können.

Klausurfokus: Hardwarenahes C

- Row- vs. Column-Major / Lokalität
- MMIO-Pointer
- `volatile` bei MMIO/IRQ (Compiler darf Zugriffe nicht wegoptimieren) — ersetzt keine `Atomics`
- Races erkennen; `_Atomic` / `stdatomic` als Lösung

Notizen

„Warum ist dieser Schnipsel gefährlich?“

Klausurfokus: System

- Polling vs. IRQ; CLINT vs. PLIC
- Timer/Prescaler, PWM Duty Cycle; WDT
- UART/SPI/I2C-Eigenschaften
- DMA: SRC, DST, Increment
- OoO: Renaming, ROB (Grundidee)

Notizen

Display: SPI oder I2C — und warum?

Was nicht im Fokus steht

- Keine auswendig gelernten Chip-Basisadressen (ggf. gegeben)
- Keine langen Assembler-Programme (MCT1)
- Keine Halbleiterphysik/Leckstrom-Formeln

Notizen

Konzepte und Prinziprechnungen — keine Datenblatt-Trivia.

Offene Fragerunde

- Welche Bausteine des SoCs sind noch unklar?
- Was soll wiederholt werden?

Notizen

Nutzen Sie die Gelegenheit vor der Klausurphase.

Zusammenfassung Block 9

- SMP: private L1, gemeinsamer Speicher → Kohärenz nötig
- MESI: M/E/S/I; Write auf Shared → Invalidate
- Race: `i++` ist RMW — MESI reicht nicht; Atomics (LR/SC, AMO)
- Amdahl: serieller Anteil begrenzt den Multicore-Speedup
- Klausur: AMAT, Mapping, IRQ/Timer, PWM, DMA, OoO-Idee, MESI

Notizen

Kohärenz hält Caches ehrlich; Atomics halten Updates unteilbar; Amdahl hält Erwartungen realistisch.

Abschluss und Ausblick

- Theorie MCT3 abgeschlossen
- Labor-Finale: Probeaufgaben, Renode, offene Fragen

Vielen Dank für Ihre Mitarbeit im Semester!

Notizen

Quellen: RISC-V A-Extension und RVWMO, C11-Atomics, Kernel-Hinweis zu False Sharing. Kein kohärenter Multicore-Simulator und keine RISC-V-Toolchain wurden für Zeit oder Assemblat verwendet. Freude, Sie durch die SoC-Ebenen begleitet zu haben — viel Erfolg in der Klausur.